02 — pg_stat_* Views: Comprehensive Reference for Production Monitoring

Table of Contents

Learning Objectives

By the end of this section you will be able to:

- Query every major `pg_stat_*` view and interpret each important column
- Write production-ready monitoring queries covering connections, locks, cache, bloat, replication, and WAL
- Explain why statistics are cumulative and how to reset them safely
- Understand wait events exposed by `pg_stat_activity` and diagnose them
- Use `pg_stat_statements` for historical query performance analysis
- Correlate data across multiple `pg_stat_*` views for holistic diagnostics

---

Overview of pg_stat_* Views

PostgreSQL exposes a rich set of statistics collector views under the `pg_catalog` schema. They are divided into:

Category	Views	Reset Function
Process activity	pg_stat_activity	n/a (live data)
Database-wide	pg_stat_database, pg_stat_bgwriter, pg_stat_wal	pg_stat_reset(), pg_stat_reset_shared()
Table-level	pg_stat_user_tables, pg_statio_user_tables	pg_stat_reset()

Index-level	pg_stat_user_indexes, pg_statio_user_indexes	pg_stat_reset()
Replication	pg_stat_replication, pg_stat_replication_slots	n/a (live data)
Extension	pg_stat_statements	pg_stat_statements_reset()

Important: Most counters are cumulative since the last statistics reset (`stats_reset` in `pg_stat_database`). Rates require comparing two snapshots over time.

---

pg_stat_activity

One row per server process (backend or autovacuum worker). This is the single most useful view for real-time diagnostics.

Key Columns

Column	Type	Description
`pid`	integer	OS process ID of the backend
`datname`	name	Name of the database the process is connected to
`usename`	name	Username of the connected client
`application_name`	text	Identifier set by the client application
`client_addr`	inet	Client IP address (NULL for Unix socket)
`backend_start`	timestamptz	When this backend connected
`xact_start`	timestamptz	Start of current transaction (NULL if no active transaction)
`query_start`	timestamptz	Start of the current or last query
`state_change`	timestamptz	When `state` last changed
`wait_event_type`	text	Category of wait event (Lock, LWLock, IO, Client, etc.)
`wait_event`	text	Specific wait event name
`state`	text	Session state: active / idle / idle in transaction / idle in transaction (aborted) / fastpath function call / disabled
`query`	text	Current or most recent SQL statement
`backend_type`	text	Process type: client backend / autovacuum launcher / autovacuum worker / logical replication worker / etc.

Wait Event Types

Type	Meaning	Common Causes
`Lock`	Waiting for a heavyweight lock	DDL operations, explicit LOCK TABLE

`LWLock`	Waiting for a lightweight lock	Buffer pin contention, WAL write
`IO`	Waiting for disk I/O	Buffer cache miss, checkpoint, WAL write
`Client`	Waiting for client to send data	Idle connection, slow network
`IPC`	Inter-process communication wait	Parallel query coordination
`Timeout`	Waiting for a timeout	pg_sleep(), lock_timeout
`Activity`	Background process waiting for work	Autovacuum launcher, bgwriter idle
`Extension`	Wait from an extension	pg_wait_sampling, etc.

Useful State Combinations

```
state = 'active' AND wait_event IS NULL          → CPU-bound query (executing
normally)
state = 'active' AND wait_event_type = 'Lock'    → blocked on heavyweight lock
state = 'active' AND wait_event_type = 'IO'      → reading/writing disk
state = 'idle in transaction'                     → transaction open, not executing
state = 'idle'                                    → connected, not in transaction
```

---

pg_stat_bgwriter

One row for the entire cluster. Tracks the background writer and checkpoint activity.

Key Columns

Column	Description
`checkpoints_timed`	Checkpoints triggered by `checkpoint_timeout`
`checkpoints_req`	Checkpoints triggered by `max_wal_size` being reached
`checkpoint_write_time`	Milliseconds spent writing files during checkpoints
`checkpoint_sync_time`	Milliseconds spent syncing files during checkpoints
`buffers_checkpoint`	Buffers written during checkpoints
`buffers_clean`	Buffers written by bgwriter (proactive cleaning)
`maxwritten_clean`	Times bgwriter stopped because it wrote max pages per round
`buffers_backend`	Buffers written directly by backends (bad — means bgwriter behind)
`buffers_backend_fsync`	Times a backend had to execute its own fsync call
`buffers_alloc`	Total buffers allocated
`stats_reset`	Time when these statistics were last reset

Derived Metrics

```sql
-- % of writes done by backends (should be < 5%)
buffers_backend * 100.0 / (buffers_checkpoint + buffers_clean + buffers_backend)

-- % of checkpoints that were forced (should be < 10%)
checkpoints_req * 100.0 / (checkpoints_timed + checkpoints_req)
```

---

pg_stat_database

One row per database. Provides database-wide aggregate statistics.

Key Columns

Column	Description
`datname`	Database name
`numbackends`	Current active connections
`xact_commit`	Committed transactions (cumulative)
`xact_rollback`	Rolled-back transactions (cumulative)
`blks_read`	Disk blocks read (cache miss)
`blks_hit`	Blocks found in shared_buffers (cache hit)
`tup_returned`	Rows returned by queries
`tup_fetched`	Rows fetched (selected) by queries
`tup_inserted`	Rows inserted
`tup_updated`	Rows updated
`tup_deleted`	Rows deleted
`conflicts`	Queries canceled due to recovery conflicts (replicas)
`temp_files`	Temporary files created (hash/sort spills)
`temp_bytes`	Total bytes written to temp files
`deadlocks`	Deadlocks detected
`blk_read_time`	Milliseconds spent reading disk blocks (requires `track_io_timing = on`)
`blk_write_time`	Milliseconds spent writing disk blocks
`stats_reset`	When stats were last reset

---

pg_stat_user_tables

One row per user table. The primary view for table health, autovacuum tracking, and bloat monitoring.

Key Columns

Column	Description
`schemaname`	Schema name
`relname`	Table name
`seq_scan`	Number of sequential (full table) scans
`seq_tup_read`	Rows read by sequential scans
`idx_scan`	Number of index scans
`idx_tup_fetch`	Rows fetched by index scans
`n_tup_ins`	Rows inserted
`n_tup_upd`	Rows updated
`n_tup_del`	Rows deleted
`n_tup_hot_upd`	HOT (Heap-Only Tuple) updates — in-place updates without index churn
`n_live_tup`	Estimated live rows
`n_dead_tup`	Estimated dead rows (need VACUUM)
`n_mod_since_analyze`	Rows modified since last ANALYZE
`n_ins_since_vacuum`	Rows inserted since last VACUUM
`last_vacuum`	Last time VACUUM ran manually
`last_autovacuum`	Last time autovacuum ran
`last_analyze`	Last time ANALYZE ran manually
`last_autoanalyze`	Last time autoanalyze ran
`vacuum_count`	Number of manual VACUUMs
`autovacuum_count`	Number of autovacuums
`analyze_count`	Number of manual ANALYZEs
`autoanalyze_count`	Number of autoanalyzes

Key Derived Metrics

```
dead_tuple_ratio = n_dead_tup / (n_live_tup + n_dead_tup)  -- > 0.20 needs vacuum
seq_vs_idx_ratio = seq_scan / (seq_scan + idx_scan)          -- high = missing index
hot_update_ratio = n_tup_hot_upd / n_tup_upd                 -- low = high index
overhead
```

---

pg_stat_user_indexes

One row per user index. Used to identify unused and redundant indexes.

Key Columns

Column	Description
`schemaname`	Schema
`relname`	Table the index belongs to
`indexrelname`	Index name
`idx_scan`	Number of times this index was used in a scan
`idx_tup_read`	Rows read via this index
`idx_tup_fetch`	Rows actually fetched (after heap visibility check)

Key Query: Find Unused Indexes

```
SELECT
    schemaname,
    relname,
    indexrelname,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
  AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_relation_size(indexrelid) DESC;
```

---

pg_statio_user_tables

One row per user table. Tracks I/O at the buffer/disk level — complements `pg_stat_user_tables`.

Key Columns

Column	Description
`heap_blks_read`	Blocks read from disk for the table (cache miss)
`heap_blks_hit`	Blocks found in shared_buffers for the table (cache hit)
`idx_blks_read`	Blocks read from disk for all indexes on this table
`idx_blks_hit`	Index blocks found in cache
`toast_blks_read`	TOAST table disk reads
`toast_blks_hit`	TOAST table cache hits

`tidx_blks_read`	TOAST index disk reads
`tidx_blks_hit`	TOAST index cache hits

Table-Level Cache Hit Ratio

```
SELECT
    schemaname,
    relname,
    heap_blks_hit + heap_blks_read AS total_reads,
    ROUND(heap_blks_hit * 100.0 / NULLIF(heap_blks_hit + heap_blks_read, 0), 2) AS
table_cache_hit_ratio,
    ROUND(idx_blks_hit * 100.0 / NULLIF(idx_blks_hit + idx_blks_read, 0), 2) AS
index_cache_hit_ratio
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 1000
ORDER BY heap_blks_read DESC
LIMIT 20;
```

---

pg_stat_replication

One row per WAL sender (standby connection). Only visible on the primary.

Key Columns

Column	Description
`pid`	WAL sender process ID
`usename`	Username for replication connection
`application_name`	Standby's `application_name`
`client_addr`	Standby's IP address
`state`	`startup`, `catchup`, `streaming`, `backup`, `stopping`
`sent_lsn`	LSN sent to standby
`write_lsn`	LSN written to standby's disk
`flush_lsn`	LSN flushed (fsync'd) on standby
`replay_lsn`	LSN applied to standby's data
`write_lag`	Time delay between primary flush and standby write
`flush_lag`	Time delay between primary flush and standby flush
`replay_lag`	Time delay between primary flush and standby replay
`sync_state`	`async`, `potential`, `sync`, `quorum`

LSN Lag Calculation

```
-- Byte lag (accurate even without time lag columns)
SELECT
    application_name,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS byte_lag,
    replay_lag
FROM pg_stat_replication;
```

---

pg_stat_wal

One row for the entire cluster (PostgreSQL 14+). Tracks WAL generation and write statistics.

Key Columns

Column	Description
`wal_records`	Number of WAL records generated
`wal_fpi`	Full page images written (increases after checkpoints)
`wal_bytes`	Total bytes of WAL generated
`wal_buffers_full`	Times WAL was written because WAL buffers were full
`wal_write`	Number of WAL writes to disk
`wal_sync`	Number of WAL syncs (fsync calls)
`wal_write_time`	Milliseconds spent writing WAL (requires `track_wal_io_timing = on`)
`wal_sync_time`	Milliseconds spent syncing WAL
`stats_reset`	When stats were last reset

---

pg_stat_statements (Extension)

Requires `shared_preload_libraries = 'pg_stat_statements'` and `CREATE EXTENSION pg_stat_statements`.

One row per unique query fingerprint (constants normalized). The most important view for query-level performance analysis.

Key Columns

Column	Description
`userid`	User who ran the query
`dbid`	Database OID
`queryid`	Normalized query hash

`query`	Normalized query text (constants replaced with $1, $2, etc.)
`calls`	Number of times executed
`total_exec_time`	Total execution time in milliseconds
`mean_exec_time`	Average execution time per call
`min_exec_time`	Minimum execution time
`max_exec_time`	Maximum execution time
`stddev_exec_time`	Standard deviation of execution time
`rows`	Total rows returned/affected
`shared_blks_hit`	Shared buffer hits
`shared_blks_read`	Shared buffer reads (disk hits)
`shared_blks_dirtied`	Buffers dirtied
`shared_blks_written`	Buffers written to disk
`local_blks_hit`	Local buffer hits (temp tables)
`temp_blks_read`	Temp file blocks read (sort/hash spill)
`temp_blks_written`	Temp file blocks written
`blk_read_time`	Time spent reading blocks (requires track_io_timing)
`blk_write_time`	Time spent writing blocks

---

pg_locks

One row per lock or lock request. Used for lock contention diagnosis.

Key Columns

Column	Description
`locktype`	`relation`, `transactionid`, `tuple`, `advisory`, etc.
`database`	Database OID
`relation`	OID of the locked relation (for `relation` locktype)
`transactionid`	Transaction holding/waiting for a transaction lock
`classid`, `objid`	For advisory locks or system catalog locks
`mode`	Lock mode (AccessShareLock, RowExclusiveLock, AccessExclusiveLock, etc.)
`granted`	`true` if lock is held; `false` if waiting
`pid`	Backend holding or waiting for the lock

`fastpath`	Whether lock was acquired via fast path (lighter weight)

25+ Production Monitoring Queries

Q1: Active Connection Count vs Limit

```
SELECT
    COUNT(*) AS total_connections,
    SUM(CASE WHEN state = 'active' THEN 1 ELSE 0 END) AS active,
    SUM(CASE WHEN state = 'idle' THEN 1 ELSE 0 END) AS idle,
    SUM(CASE WHEN state LIKE 'idle in transaction%' THEN 1 ELSE 0 END) AS
idle_in_txn,
    current_setting('max_connections')::INT AS max_allowed,
    ROUND(COUNT(*) * 100.0 / current_setting('max_connections')::INT, 1) AS pct_used
FROM pg_stat_activity
WHERE pid <> pg_backend_pid();
```

Q2: Oldest Transactions and Idle-in-Transaction Sessions

```
SELECT
    pid,
    datname,
    usename,
    application_name,
    state,
    EXTRACT(EPOCH FROM (now() - xact_start))::INT AS txn_age_seconds,
    LEFT(query, 120) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY txn_age_seconds DESC NULLS LAST
LIMIT 10;
```

Q3: Full Lock Blocking Tree

```
WITH RECURSIVE lock_tree AS (
    -- Base: blocked sessions
    SELECT
        blocked.pid AS blocked_pid,
        blocked.query AS blocked_query,
        blocking.pid AS blocking_pid,
        blocking.query AS blocking_query,
        1 AS depth
    FROM pg_stat_activity AS blocked
    JOIN pg_stat_activity AS blocking
        ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
    WHERE pg_blocking_pids(blocked.pid) <> '{}'::INT[]
)
```

```
SELECT
    blocked_pid,
    LEFT(blocked_query, 80) AS blocked_query,
    blocking_pid,
    LEFT(blocking_query, 80) AS blocking_query,
    depth
FROM lock_tree
ORDER BY depth, blocking_pid;
```

Q4: Wait Events Distribution

```
SELECT
    wait_event_type,
    wait_event,
    COUNT(*) AS sessions_waiting,
    string_agg(DISTINCT state, ', ') AS states
FROM pg_stat_activity
WHERE wait_event IS NOT NULL
  AND pid <> pg_backend_pid()
GROUP BY wait_event_type, wait_event
ORDER BY sessions_waiting DESC;
```

Q5: Top 10 Queries by Total Time (pg_stat_statements)

```
SELECT
    LEFT(query, 100) AS query,
    calls,
    ROUND(total_exec_time::NUMERIC, 2) AS total_ms,
    ROUND(mean_exec_time::NUMERIC, 2) AS mean_ms,
    ROUND(stddev_exec_time::NUMERIC, 2) AS stddev_ms,
    rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Q6: Top 10 Queries by Mean Execution Time

```
SELECT
    LEFT(query, 100) AS query,
    calls,
    ROUND(mean_exec_time::NUMERIC, 2) AS mean_ms,
    ROUND(max_exec_time::NUMERIC, 2) AS max_ms,
    ROUND(stddev_exec_time::NUMERIC, 2) AS stddev_ms
FROM pg_stat_statements
WHERE calls >= 100  -- only queries with statistical significance
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Q7: Queries with Highest I/O (Cache Misses)

```
SELECT
    LEFT(query, 100) AS query,
    calls,
    shared_blks_read AS disk_reads,
    shared_blks_hit AS cache_hits,
    ROUND(shared_blks_read * 100.0 / NULLIF(shared_blks_read + shared_blks_hit, 0),
2) AS miss_rate_pct
FROM pg_stat_statements
WHERE shared_blks_read + shared_blks_hit > 1000
ORDER BY disk_reads DESC
LIMIT 10;
```

Q8: Queries Spilling to Temp Files

```
SELECT
    LEFT(query, 100) AS query,
    calls,
    temp_blks_written,
    temp_blks_read,
    ROUND(mean_exec_time::NUMERIC, 2) AS mean_ms
FROM pg_stat_statements
WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 10;
```

Q9: Table Sequential Scans — Missing Index Candidates

```
SELECT
    schemaname,
    relname,
    seq_scan,
    seq_tup_read,
    idx_scan,
    ROUND(seq_scan * 100.0 / NULLIF(seq_scan + idx_scan, 0), 2) AS seq_scan_pct,
    n_live_tup,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
WHERE seq_scan > 0
  AND n_live_tup > 10000
ORDER BY seq_scan DESC
LIMIT 20;
```

Q10: Tables Most in Need of VACUUM

```sql
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    ROUND(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
    last_autovacuum,
    autovacuum_count,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;
```

Q11: Tables Most in Need of ANALYZE

```sql
SELECT
    schemaname,
    relname,
    n_mod_since_analyze,
    n_live_tup,
    ROUND(n_mod_since_analyze * 100.0 / NULLIF(n_live_tup, 0), 2) AS mod_pct,
    last_autoanalyze,
    last_analyze
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 1000
ORDER BY n_mod_since_analyze DESC
LIMIT 20;
```

Q12: Index Usage — Unused Indexes

```sql
SELECT
    i.schemaname,
    i.relname AS table_name,
    i.indexrelname AS index_name,
    i.idx_scan,
    pg_size_pretty(pg_relation_size(i.indexrelid)) AS index_size,
    ix.indisunique AS is_unique
FROM pg_stat_user_indexes i
JOIN pg_index ix ON ix.indexrelid = i.indexrelid
WHERE i.idx_scan = 0
  AND NOT ix.indisprimary
  AND NOT ix.indisunique
ORDER BY pg_relation_size(i.indexrelid) DESC;
```

Q13: Duplicate and Redundant Indexes

```sql
SELECT
    a.schemaname,
```

```
    a.relname AS table_name,
    a.indexrelname AS index_a,
    b.indexrelname AS index_b,
    pg_get_indexdef(a.indexrelid) AS definition_a,
    pg_get_indexdef(b.indexrelid) AS definition_b
FROM pg_stat_user_indexes a
JOIN pg_stat_user_indexes b
    ON a.relid = b.relid
    AND a.indexrelid < b.indexrelid
WHERE pg_get_indexdef(a.indexrelid) = pg_get_indexdef(b.indexrelid);
```

Q14: bgwriter Health

```
SELECT
    checkpoints_timed,
    checkpoints_req,
    ROUND(checkpoints_req * 100.0 / NULLIF(checkpoints_timed + checkpoints_req, 0),
2) AS forced_pct,
    ROUND(checkpoint_write_time / 1000.0 / NULLIF(checkpoints_timed +
checkpoints_req, 0), 2) AS avg_write_s,
    ROUND(checkpoint_sync_time / 1000.0 / NULLIF(checkpoints_timed +
checkpoints_req, 0), 2) AS avg_sync_s,
    buffers_checkpoint,
    buffers_clean,
    maxwritten_clean,
    buffers_backend,
    ROUND(buffers_backend * 100.0 / NULLIF(buffers_checkpoint + buffers_clean +
buffers_backend, 0), 2) AS backend_write_pct,
    pg_size_pretty(buffers_alloc * 8192) AS total_allocated,
    stats_reset
FROM pg_stat_bgwriter;
```

Q15: Replication Status and Lag

```
SELECT
    pid,
    application_name,
    client_addr,
    state,
    sync_state,
    pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn) AS unsent_bytes,
    pg_wal_lsn_diff(sent_lsn, flush_lsn) AS unflushed_bytes,
    pg_wal_lsn_diff(flush_lsn, replay_lsn) AS unreplayed_bytes,
    pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS total_lag_bytes,
    write_lag,
    flush_lag,
    replay_lag
FROM pg_stat_replication
ORDER BY total_lag_bytes DESC NULLS LAST;
```

Q16: Replication Slot Monitoring

```
SELECT
    slot_name,
    plugin,
    slot_type,
    database,
    active,
    active_pid,
    pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS lag_bytes,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS
lag_pretty,
    confirmed_flush_lsn,
    restart_lsn
FROM pg_replication_slots
ORDER BY lag_bytes DESC NULLS LAST;
```

Q17: Database Activity Rates (Per-Second with Two Snapshots)

```
-- Run once, wait 60 seconds, run again, compare
-- Snapshot approach: store results in a temp table
CREATE TEMP TABLE stats_snap AS
SELECT
    datname,
    xact_commit,
    xact_rollback,
    blks_read,
    blks_hit,
    tup_inserted,
    tup_updated,
    tup_deleted,
    now() AS snapshot_time
FROM pg_stat_database
WHERE datname = current_database();
```

Q18: XID Age — Wraparound Emergency Check

```
SELECT
    datname,
    age(datfrozenxid) AS db_xid_age,
    2147483647 - age(datfrozenxid) AS xids_remaining,
    ROUND(age(datfrozenxid) * 100.0 / 2147483647, 2) AS pct_toward_wraparound
FROM pg_database
ORDER BY db_xid_age DESC;
```

Q19: Table-Level XID Age

```sql
SELECT
    n.nspname AS schema,
    c.relname AS table_name,
    age(c.relfrozenxid) AS xid_age,
    ROUND(age(c.relfrozenxid) * 100.0 / 2147483647, 2) AS pct_of_max,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS table_size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
  AND n.nspname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
ORDER BY xid_age DESC
LIMIT 20;
```

Q20: Table Size Leaderboard

```sql
SELECT
    schemaname,
    relname,
    pg_size_pretty(pg_relation_size(relid)) AS table_size,
    pg_size_pretty(pg_indexes_size(relid)) AS indexes_size,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
    pg_size_pretty(pg_total_relation_size(relid) - pg_relation_size(relid) -
pg_indexes_size(relid)) AS toast_size
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 20;
```

Q21: HOT Update Effectiveness

```sql
-- HOT updates avoid index maintenance; low ratio means indexes are too wide
SELECT
    schemaname,
    relname,
    n_tup_upd AS total_updates,
    n_tup_hot_upd AS hot_updates,
    ROUND(n_tup_hot_upd * 100.0 / NULLIF(n_tup_upd, 0), 2) AS hot_update_pct,
    n_live_tup,
    n_dead_tup
FROM pg_stat_user_tables
WHERE n_tup_upd > 1000
ORDER BY hot_update_pct ASC
LIMIT 20;
```

Q22: pg_statio — Per-Table I/O Cache Ratio

```sql
SELECT
    schemaname,
    relname,
```

```
    heap_blks_hit,
    heap_blks_read,
    ROUND(heap_blks_hit * 100.0 / NULLIF(heap_blks_hit + heap_blks_read, 0), 2) AS
table_cache_pct,
    idx_blks_hit,
    idx_blks_read,
    ROUND(idx_blks_hit * 100.0 / NULLIF(idx_blks_hit + idx_blks_read, 0), 2) AS
index_cache_pct
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 10000
ORDER BY heap_blks_read DESC
LIMIT 20;
```

Q23: Deadlock Detection History

```
-- Deadlock count per database (from pg_stat_database)
SELECT
    datname,
    deadlocks,
    xact_commit,
    xact_rollback,
    ROUND(deadlocks * 1000.0 / NULLIF(xact_commit, 0), 4) AS
deadlocks_per_1000_commits
FROM pg_stat_database
WHERE datname NOT IN ('template0', 'template1')
ORDER BY deadlocks DESC;
```

Q24: All Current Locks (Simplified)

```
SELECT
    l.pid,
    a.state,
    a.wait_event_type,
    a.wait_event,
    l.locktype,
    l.mode,
    l.granted,
    CASE
        WHEN l.relation IS NOT NULL THEN c.relname
        ELSE NULL
    END AS object_name,
    LEFT(a.query, 80) AS query
FROM pg_locks l
LEFT JOIN pg_stat_activity a ON a.pid = l.pid
LEFT JOIN pg_class c ON c.oid = l.relation
WHERE l.pid <> pg_backend_pid()
ORDER BY l.granted, l.pid;
```

Q25: Statistics Reset Timestamp Check

```sql
-- Know when your stats were last reset
SELECT
    'pg_stat_database' AS view_name,
    stats_reset
FROM pg_stat_database
WHERE datname = current_database()
UNION ALL
SELECT
    'pg_stat_bgwriter',
    stats_reset
FROM pg_stat_bgwriter
UNION ALL
SELECT
    'pg_stat_wal' AS view_name,
    stats_reset
FROM pg_stat_wal;
```

Q26: Long-Running Autovacuum Workers

```sql
SELECT
    pid,
    datname,
    EXTRACT(EPOCH FROM (now() - query_start))::INT AS running_seconds,
    LEFT(query, 200) AS autovacuum_query,
    wait_event_type,
    wait_event
FROM pg_stat_activity
WHERE backend_type = 'autovacuum worker'
ORDER BY running_seconds DESC;
```

Q27: Connection Age Distribution

```sql
-- How old are the current connections?
SELECT
    datname,
    application_name,
    COUNT(*) AS count,
    MIN(EXTRACT(EPOCH FROM (now() - backend_start))::INT) AS min_age_sec,
    MAX(EXTRACT(EPOCH FROM (now() - backend_start))::INT) AS max_age_sec,
    ROUND(AVG(EXTRACT(EPOCH FROM (now() - backend_start))::INT)) AS avg_age_sec
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY datname, application_name
ORDER BY count DESC;
```

Q28: WAL Generation Rate

```sql
-- Requires PostgreSQL 14+ for pg_stat_wal
SELECT
    wal_records,
    wal_fpi,
    pg_size_pretty(wal_bytes) AS wal_generated,
    wal_buffers_full,
    wal_write,
    wal_sync,
    ROUND(wal_write_time / 1000.0, 2) AS write_time_s,
    ROUND(wal_sync_time / 1000.0, 2) AS sync_time_s,
    stats_reset
FROM pg_stat_wal;
```

Common Mistakes

1. **Treating cumulative counters as rates.** `xact_commit` is a total-since-reset. Always compare two snapshots with a known time delta to compute TPS.
2. **Querying `pg_stat_activity` for one-time diagnostics only.** These views reflect point-in-time state. A query that runs for 10 seconds and holds a lock may already be gone by the time you look. Continuous collection into a time-series database (Prometheus) is necessary for post-incident analysis.
3. **Not enabling `pg_stat_statements`.** Without this extension, you have no historical record of which queries ran slowly. Enable it at PostgreSQL startup; it cannot be added dynamically without a restart.
4. **Not setting `track_io_timing = on`.** The `blk_read_time` and `blk_write_time` columns in `pg_stat_database` and `pg_stat_statements` are zero unless this GUC is enabled. It has a small overhead (~1% on SSDs) but is essential for I/O diagnostics.
5. **Ignoring `n_mod_since_analyze`.** High values in this column mean the planner is working with stale statistics, which causes bad query plans. Autovacuum triggers analyze automatically, but busy tables may need manual intervention.
6. **Using `pg_stat_reset()` without documenting it.** Resetting all statistics silently discards all accumulated data. If you reset during an incident investigation, you lose the evidence. Always document when and why you reset.
7. **Checking only `pg_stat_replication`** without checking `pg_replication_slots`. Slots can accumulate lag even when no streaming replication is active.

Best Practices

- **Enable `pg_stat_statements` on all production databases.** Set `pg_stat_statements.max = 10000` and `pg_stat_statements.track = all` in `postgresql.conf`.
- **Enable `track_io_timing = on`** for accurate I/O timing in `pg_stat_statements`.
- **Collect `pg_stat_activity` every 15 seconds** into a time-series store so you can replay connection state during post-incident analysis.
- **Join `pg_stat_user_tables` with `pg_class`** to get OIDs needed for `pg_relation_size()`.

- **Use `pg_stat_reset_shared('bgwriter')`** to reset only bgwriter stats without affecting database stats, when you need a clean measurement window.
- **Monitor `pg_replication_slots.active` and lag** every minute. Drop inactive slots that are no longer needed.
- **Schedule periodic captures** of the views to a monitoring table. This gives you point-in-time history even when the original statistics are reset.

Interview Questions and Answers

Q1: What does `n_dead_tup` in `pg_stat_user_tables` represent and what should you do when it is high?

A: `n_dead_tup` is PostgreSQL's estimate of the number of dead (obsolete) row versions in the table heap. Dead tuples are created by `UPDATE` and `DELETE` operations — the old version of a row is not immediately removed but is marked as invisible to all future transactions. A high dead tuple count (>20% of total rows) indicates that VACUUM has not run recently or is failing to keep up. The fix is to run `VACUUM <table>` manually or tune autovacuum parameters for that table (`autovacuum_vacuum_scale_factor` , `autovacuum_vacuum_cost_delay`).

Q2: What is the difference between `idx_scan` in `pg_stat_user_tables` vs `pg_stat_user_indexes` ?

A: In `pg_stat_user_tables` , `idx_scan` is the total number of index scans performed on the table — the sum across all indexes. In `pg_stat_user_indexes` , `idx_scan` is per individual index. You use the table-level view to see the ratio of sequential to index scans, and the index-level view to identify which specific indexes are being used and which are dead weight (idx_scan = 0).

Q3: A session appears in `pg_stat_activity` with `state = 'active'` and `wait_event_type = 'Lock'` . What exactly is happening and what do you do?

A: The session is actively running but blocked waiting for a heavyweight lock (e.g., a table-level lock held by another transaction). It is not consuming CPU — it is parked in the kernel waiting. To investigate: query `pg_blocking_pids(pid)` to find the blocking PIDs, then look at their `query` in `pg_stat_activity` . You have three options: (1) wait for the blocking transaction to complete, (2) `SELECT pg_terminate_backend(blocking_pid)` to kill the blocker, or (3) investigate why the blocking query is holding the lock so long and fix the application logic.

Q4: How do you calculate transactions per second (TPS) from `pg_stat_database` ?

A: Because `xact_commit` and `xact_rollback` are cumulative counters, you need two snapshots:

```
-- snapshot1 at time T1, snapshot2 at time T2
TPS = (snap2.xact_commit - snap1.xact_commit + snap2.xact_rollback -
snap1.xact_rollback)
      / EXTRACT(EPOCH FROM (T2 - T1))
```

Tools like `postgres_exporter` handle this automatically by computing the rate between scrapes.

Q5: What does `wal_buffers_full` in `pg_stat_wal` indicate and how do you fix it?

A: `wal_buffers_full` counts how many times WAL was written to disk because WAL buffers were full (all `wal_buffers` slots were occupied). This indicates `wal_buffers` is too small. The fix is to increase `wal_buffers` in `postgresql.conf` . The recommended default is `max(64kB, 3% of`

`shared_buffers)`, but high-write workloads benefit from values like 64MB. Note that this requires a PostgreSQL restart.

Q6: How would you find the 5 slowest queries currently executing in PostgreSQL?

A: Query `pg_stat_activity` filtering for `state = 'active'` and ordering by `query_start ASC` (oldest = slowest):

```
SELECT pid, usename, datname,
       EXTRACT(EPOCH FROM (now() - query_start))::INT AS duration_s,
       wait_event_type, wait_event, LEFT(query, 150)
FROM pg_stat_activity
WHERE state = 'active' AND query_start IS NOT NULL
ORDER BY query_start ASC LIMIT 5;
```

For historical data (not just currently executing), use `pg_stat_statements` ordered by `mean_exec_time DESC` or `total_exec_time DESC`.

Q7: What is the purpose of `pg_statio_user_tables` vs `pg_stat_user_tables` and when do you use each?

A: `pg_stat_user_tables` tracks logical operations: scans, rows inserted/updated/deleted, vacuum/analyze timestamps, dead tuples. `pg_statio_user_tables` tracks physical I/O: buffer cache hits and disk reads for the table heap and its associated indexes. You use `pg_stat_user_tables` to understand access patterns and maintenance health. You use `pg_statio_user_tables` to understand cache efficiency — if `heap_blks_read` is high relative to `heap_blks_hit`, that table is causing many disk reads and needs either more `shared_buffers`, an index to reduce row scans, or both.

Q8: An index has `idx_scan = 0` since the last statistics reset. Should you drop it immediately?

A: Not necessarily. Consider: (1) When was `stats_reset`? If reset an hour ago, zero scans means nothing. (2) Is the index used only occasionally (e.g., for monthly reports)? (3) Is it a unique constraint or primary key (these must be kept regardless of scan count)? (4) Is there significant write traffic? Unused indexes still slow down `INSERT`, `UPDATE`, and `DELETE`. The correct approach: verify the index has been unused for at least 2-4 weeks of normal production traffic before dropping, and always drop with a rollback plan (`CREATE INDEX CONCURRENTLY` to recreate if needed).

Cross-References

- `01_monitoring_overview.md` — signals and alert thresholds that these views feed
- `04_observability_stack.md` — how postgres_exporter scrapes these views for Prometheus
- `06_maintenance_procedures.md` — using `pg_stat_user_tables` to drive VACUUM and ANALYZE decisions
- `07_incident_response.md` — incident-specific queries that use these views
- `09_troubleshooting_guide.md` — systematic use of these views for diagnosis